

ESTRUTURAS DE DADOS II

GAME DESIGN DOCUMENT BRIEFING

Design de Jogos Educativos sobre Árvores Avançadas

Disciplina	Estruturas de Dados II — Ciência da Computação
Tema	Design de Jogos Educativos sobre Árvores Avançadas por meio da Engenharia Reversa e Reuso de Modelos
Integrantes	
Data / Turma	11 / 09 / 2026 Turma: Ciência da Computação

1. Identificação do Grupo

Integrantes do Grupo: Lucas Yan Santana Caetano

Data: 11 / 09 / 2026 Turma: Ciência da Computação

2. Orientações Gerais e Objetivos

Esta atividade propõe a análise de um modelo existente de visualização/aprendizagem de estruturas de dados e a elaboração de uma proposta de melhoria pedagógica. A proposta abaixo reutiliza como referência o VisuAlgo, ferramenta interativa de visualização de estruturas e algoritmos, e especifica uma mecânica educativa baseada em Árvore AVL. Não se propõe a implementação/programação de um jogo; o objetivo é documentar a mecânica e relacionar os conceitos de Estruturas de Dados II ao gameplay.

3. Estrutura e Roteiro do Workshop

Fase 1 — Mapeamento e Engenharia Reversa: identificação do modelo base e análise de suas limitações.

Fase 2 — Elaboração do Game Design Document Briefing: especificação da melhoria e tradução dos conceitos de ED II para as regras propostas.

Fase 3 — Preparação e Pitch: defesa oral da proposta em aproximadamente 3 minutos.

4. Template de Especificação: Game Design Document Briefing

4.1 Diagnóstico do Modelo Reutilizado

Nome do Jogo/Ferramenta Base Identificado: VisuAlgo — Binary Search Tree / AVL Tree.

Limitação Didática ou Técnica Identificada no Modelo Original: o VisuAlgo é uma ferramenta de visualização e exploração interativa, não um jogo estruturado em desafios com progressão, pontuação e condições de vitória/derrota. Ele permite ao estudante executar operações como Search, Insert e Remove e acompanhar a animação e o pseudocódigo, mas a proposta aqui busca transformar essa exploração em uma atividade mais ativa: o estudante recebe uma sequência de chaves, precisa prever o desequilíbrio, calcular o fator de balanceamento, identificar o caso de rotação e escolher a correção adequada. Assim, a melhoria não altera o conceito matemático da AVL; ela acrescenta uma camada de desafio e feedback para tornar o estudante responsável pela tomada de decisão.

4.2 Seleção da Estrutura de ED II para o Upgrade

☒ Árvore AVL

☐ Árvore Rubro-Negra

☐ Árvore B / B+

Justificativa: a Árvore AVL foi selecionada porque permite transformar diretamente conceitos centrais de ED II — altura, fator de balanceamento, inserção e rotações — em decisões do

jogador. Em uma AVL, o balanceamento é mantido por rotações após inserções/remoções quando necessário.

4.3 Mapeamento de Conceitos em Mecânicas de Gameplay

Conceito Teórico de ED II	Elemento / Mecânica Correspondente no Jogo
Nó da Árvore / Chave	Cada chave recebida pelo jogador representa um nó que deve ser inserido na Árvore AVL. O valor da chave determina sua posição de acordo com a propriedade de uma BST: menores ficam à esquerda e maiores à direita.
Altura da Árvore (h)	A altura representa o nível de profundidade ocupado pela estrutura. Quanto maior a altura, maior o risco de perda de eficiência das operações. A altura também pode funcionar como indicador visual de estabilidade do cenário.
Fator de Balanceamento (FB)	Cada nó possui um indicador de estabilidade calculado pela diferença entre as alturas das subárvores esquerda e direita. O jogador deve verificar os valores após uma inserção. Para a AVL, a condição válida é $FB \in \{-1, 0, 1\}$; valores +2 ou -2 indicam desequilíbrio e exigem correção.
Operação de Correção (Rotação)	Quando surgir um desequilíbrio, o jogador deve escolher a rotação correta: direita no caso LL, esquerda no caso RR, ou duas rotações nos casos LR e RL. A escolha incorreta gera penalidade e mantém o desafio ativo até a correção.
Inserção	O jogador recebe uma sequência de chaves e deve inseri-las respeitando a regra de busca binária. Após cada inserção, precisa verificar o caminho de retorno até a raiz para localizar o primeiro nó desequilibrado.
Eficiência / Complexidade	A pontuação valoriza a manutenção de uma árvore balanceada. A proposta relaciona a manutenção da altura da AVL à eficiência $O(\log n)$ das operações que dependem da altura, enquanto uma BST pode chegar a altura $O(n)$ no pior caso.

4.4 Regras do Core Loop e Condições do Jogo

Core Loop (O que o jogador faz repetidamente durante a partida?):

1. Recebe uma nova chave para inserir na árvore.
2. Decide a posição da chave seguindo a propriedade de uma BST.
3. Atualiza a altura dos nós afetados e verifica o fator de balanceamento.
4. Se todos os fatores estiverem entre -1 e +1, continua para a próxima chave.
5. Se aparecer $FB = +2$ ou $FB = -2$, identifica se o caso é LL, RR, LR ou RL.
6. Seleciona e executa a rotação simples ou dupla correspondente.
7. Recebe feedback sobre a decisão e segue para o próximo desafio.

Condição de Vitória: completar a sequência de desafios mantendo a propriedade da Árvore AVL em todas as etapas e realizando corretamente as rotações necessárias, com o maior número possível de acertos.

Condição de Derrota (falha associada à degradação algorítmica): o jogador perde a rodada quando não consegue corrigir o desequilíbrio de um desafio ou mantém uma configuração incompatível com a propriedade AVL. A consequência didática é mostrar que uma árvore não balanceada pode aumentar sua altura e levar operações dependentes de h para um cenário de pior caso $O(n)$.

4.5 Exemplos de Desafios e Rigor Teórico

Exemplo 1 — Caso LL: inserir 30, depois 20 e depois 10. Após a inserção de 10, o nó 30 fica com fator de balanceamento +2 e o crescimento ocorreu na subárvore esquerda da esquerda. A correção é uma rotação simples à direita em 30, resultando em 20 como raiz da subárvore, com 10 à esquerda e 30 à direita.

Exemplo 2 — Caso RR: inserir 10, depois 20 e depois 30. O nó 10 fica com fator de balanceamento -2 e o crescimento ocorreu na subárvore direita da direita. A correção é uma rotação simples à esquerda em 10.

Exemplo 3 — Caso LR: inserções que produzam crescimento na direita da subárvore esquerda exigem primeiro uma rotação à esquerda no filho e depois uma rotação à direita no nó desequilibrado.

Exemplo 4 — Caso RL: inserções que produzam crescimento na esquerda da subárvore direita exigem primeiro uma rotação à direita no filho e depois uma rotação à esquerda no nó desequilibrado.

4.6 Inovação e Objetivo Pedagógico

A proposta transforma uma visualização predominantemente exploratória em uma dinâmica de tomada de decisão. O estudante não apenas observa a rotação: precisa calcular/interpretar o fator de balanceamento, reconhecer o padrão de desequilíbrio e escolher a operação de correção. O feedback informa se a escolha preservou a propriedade da árvore. Dessa forma, a mecânica permanece centrada no algoritmo e evita que a inovação seja apenas estética.

5. Checklist de Autoavaliação do Grupo

- ☒ Identificamos e citamos o jogo/modelo base existente na literatura ou na web.
- ☒ O upgrade proposto exige a aplicação prática de conceitos de ED II (AVL).
- ☒ As propriedades algorítmicas (especialmente o Fator de Balanceamento e as rotações) estão diretamente traduzidas em mecânicas ativas de jogo.
- ☒ A condição de derrota está associada ao desequilíbrio/degradação da estrutura e ao pior caso de complexidade.
- ☒ O grupo está preparado para realizar a defesa do projeto em um Pitch de 3 minutos.

6. Pitch de Apresentação — aproximadamente 3 minutos

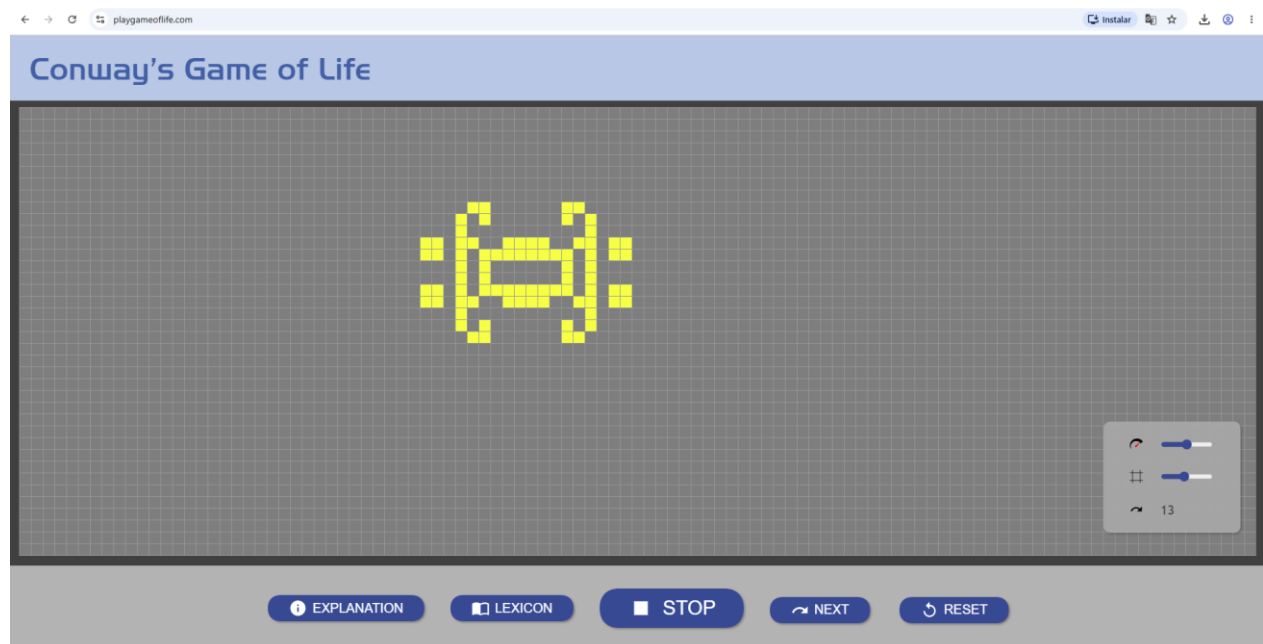
Nossa proposta parte do VisuAlgo, uma ferramenta interativa de visualização de estruturas de dados. O modelo é muito útil para visualizar árvores e acompanhar operações como busca, inserção e remoção, mas nossa proposta identifica uma limitação pedagógica: a ferramenta não é organizada como um desafio com progressão, pontuação e decisões obrigatórias do estudante.

Por isso, escolhemos a Árvore AVL como estrutura para o upgrade. A ideia é transformar o estudante em responsável pelo balanceamento da árvore. A cada rodada, ele recebe uma chave e precisa inseri-la respeitando a propriedade de uma árvore binária de busca. Depois da inserção, ele verifica a altura e o fator de balanceamento dos nós afetados.

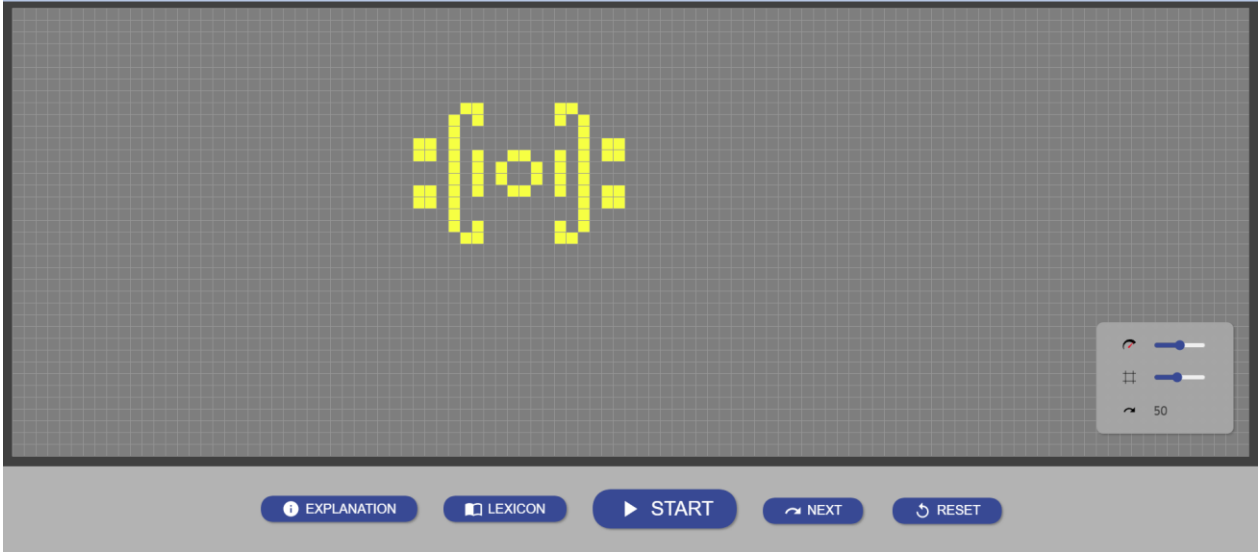
Quando o fator permanece entre -1 e 1, a árvore continua estável. Quando aparece +2 ou -2, o jogador precisa identificar o tipo de desequilíbrio: LL, RR, LR ou RL. Em seguida, deve escolher a rotação simples ou dupla correspondente. Assim, a rotação deixa de ser apenas uma animação observada e passa a ser uma decisão que o aluno precisa tomar.

A condição de vitória é concluir a sequência de desafios mantendo a propriedade AVL. A falha acontece quando o jogador não corrige corretamente o desequilíbrio, permitindo que a estrutura fique mais alta e menos eficiente. O objetivo pedagógico é fazer o aluno perceber, na prática, por que uma árvore balanceada mantém operações dependentes da altura em $O(\log n)$, enquanto uma árvore degenerada pode chegar ao pior caso $O(n)$.

Portanto, nossa proposta reutiliza um modelo existente, mas acrescenta aprendizagem ativa: o aluno observa, calcula, identifica o caso e toma a decisão algorítmica. A inovação está na mecânica pedagógica, não na criação de um software novo.



Conway's Game of Life

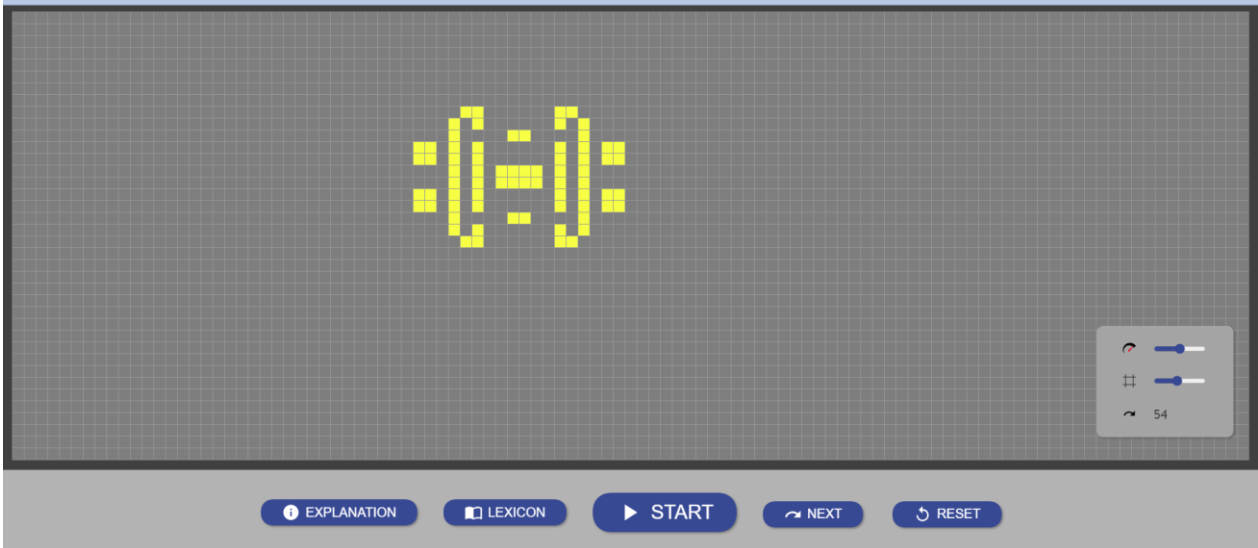


A screenshot of a Conway's Game of Life simulation. The main area is a large gray grid with a yellow blinker pattern (three cells in a vertical line) in the center. To the right of the grid is a control panel with three sliders and a counter. The counter shows the number 50. Below the grid is a navigation bar with five buttons: EXPLANATION, LEXICON, START, NEXT, and RESET.

50

EXPLANATION LEXICON START NEXT RESET

Conway's Game of Life



A screenshot of a Conway's Game of Life simulation, similar to the one above. The main area is a large gray grid with a yellow blinker pattern (three cells in a vertical line) in the center. To the right of the grid is a control panel with three sliders and a counter. The counter shows the number 54. Below the grid is a navigation bar with five buttons: EXPLANATION, LEXICON, START, NEXT, and RESET.

54

EXPLANATION LEXICON START NEXT RESET